

DBMS Unit 2 - Relational Database Design and Normalization

Structured Question Bank

PART A: SHORT ANSWER QUESTIONS (2-3 marks)

Relational Model Basics

Q1: Define a relation in the relational model.

- A relation is a table of data representing a set of tuples (rows)
- Mathematically, it is a subset of the Cartesian product of domains: $r \subseteq D_1 \times D_2 \times \dots \times D_n$
- Each tuple is an ordered list of attribute values
- Example: (Jones, Main, Harrison) $\in$ Customer relation

Q2: What is the difference between relation schema and relation instance?

- **Schema:** Structure of the relation including relation name, attributes, and domains (fixed during design)
- **Instance:** Actual data in the relation at a specific time (varies over time)
- Example: Student(RollNo, Name, Dept) is schema; actual student records are instances

Q3: What is meant by atomicity of attribute values?

- Attribute values must be simple and indivisible
- Each value should be a single value from its domain
- No sets, lists, or nested structures are allowed
- Violating atomicity leads to non-first normal form (1NF)

Q4: Define Codd's Rule 0.

- Foundation rule stating that a system must use relational capabilities exclusively
- System must be able to manage databases entirely through relational capabilities
- No other access methods should bypass relational interface

Integrity Constraints

Q5: What are domain constraints?

- Define the set of valid values for an attribute
- Ensure attribute values are within allowed limits or types
- Enforced during data input using data types and CHECK conditions

- Example: hourly_wages DECIMAL(6,2) CHECK (hourly_wages >= 400.00)

Q6: Define entity integrity.

- Primary key values must be unique and NOT NULL
- Guarantees each entity has a unique identity
- Ensures foreign keys can properly refer to valid primary keys
- Without entity integrity, foreign keys cannot reliably refer to any row

Q7: What is referential integrity?

- Foreign key values in one table must match primary key values in another table
- Prevents orphan records and maintains relationship consistency
- If a foreign key references a primary key, the referenced key must exist
- Example: account.branch_name must exist in branch.branch_name

Keys

Q8: Define superkey.

- Any set of attributes that uniquely identifies a tuple in a relation
- May include extra (redundant) attributes
- Not necessarily minimal
- Example: For Employee(ID, Name, SSN), {ID}, {ID, Name}, {SSN} are all superkeys

Q9: What is a candidate key?

- Minimal superkey with no unnecessary attributes
- Must uniquely identify every tuple in the relation
- A relation may have multiple candidate keys
- Example: {ID}, {SSN}, {Email} could all be candidate keys

Q10: Define primary key.

- One of the candidate keys chosen to uniquely identify tuples
- Must be unique and NOT NULL for all records
- Only one primary key is allowed per table
- Used as main identifier and referenced by foreign keys

Functional Dependencies

Q11: What is a functional dependency?

- Relationship where value of one attribute determines value of another
- Denoted as $A \rightarrow B$ (A determines B)
- If two tuples have same value for A, they must have same value for B
- Example: StudentID $\rightarrow$ StudentName

Q12: Differentiate between trivial and non-trivial functional dependencies.

- **Trivial FD:** Dependent attributes are subset of determinant ($X \rightarrow Y$ where $Y \subseteq X$)
- **Non-trivial FD:** Dependent attributes are not part of determinant
- Example: $\{A, B\} \rightarrow A$ is trivial; $A \rightarrow B$ is non-trivial

Q13: What is transitive dependency?

- If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$ is transitive dependency
- C is transitively dependent on A through B
- Example: EmpID $\rightarrow$ DeptID and DeptID $\rightarrow$ DeptName, so EmpID $\rightarrow$ DeptName

Normalization

Q14: Define First Normal Form (1NF).

- All attributes contain atomic (indivisible) values
- No repeating groups or arrays in any attribute
- Each cell contains single value only
- Foundation for higher normal forms

Q15: What are prime and non-prime attributes?

- **Prime attribute:** Part of a candidate key
- **Non-prime attribute:** Not part of any candidate key
- Used to define higher normal forms (2NF, 3NF)
- Critical for identifying partial and transitive dependencies

PART B: LONG ANSWER QUESTIONS (5-8 marks)

Relational Model Structure

Q1: Explain the basic structure of the relational model with examples.

Answer:

1. Mathematical Foundation

- Relation r is subset of Cartesian product: $r \subseteq D_1 \times D_2 \times \dots \times D_n$
- Each tuple is n -tuple $(a_1, a_2, \dots, a_n)$ where $a_i \in D_i$
- Tuples are unordered in a relation (no row numbers)

2. Components of Relational Structure

- **Relations:** Tables representing entities or relationships
- **Attributes:** Columns with names and domains
- **Tuples:** Rows representing individual records
- **Domains:** Set of allowed values for each attribute

3. Example Structure

Customer Relation:

Name	Street	City
Jones	Main	Harrison
Smith	North	Rye
Curry	North	Rye

4. Key Properties

- All attribute values must be atomic (indivisible)
- No duplicate tuples allowed
- Order of tuples and attributes is not significant
- Each attribute has unique name within relation

5. Representation Benefits

- Simple tabular format easy to understand
- Mathematical foundation ensures consistency
- Supports complex queries and operations
- Provides data independence and flexibility

Codd's Rules for Relational Systems

Q2: Explain Codd's Rules (any 8 rules) for relational database systems.

Answer:

1. Rule 0 - Foundation Rule

- System must use relational capabilities exclusively
- No other access methods should bypass relational interface
- All database management must be through relational operations

2. Rule 1 - Information Rule

- All data (user info + metadata) stored as values in table cells
- No hidden pointers or links between tables
- Information representation is uniform throughout system

3. Rule 2 - Guaranteed Access Rule

- Every data item accessible using table name + primary key + attribute name
- Logical addressing of all data elements
- No physical addressing or navigation required

4. Rule 3 - Systematic Treatment of NULL

- NULL values treated uniformly throughout database
- Represents missing or unknown information
- Supported in all data types and operations consistently

5. Rule 4 - Active Online Catalog

- Database description (metadata) stored in tables
- Catalog accessible using same query language as user data
- System tables contain schema information

6. Rule 8 - Physical Data Independence

- Changes in physical storage don't affect applications
- Applications immune to storage structure changes
- Logical view remains constant despite physical changes

7. Rule 9 - Logical Data Independence

- Changes in table structure don't require application changes
- Views provide interface stability
- Schema evolution doesn't break existing applications

8. Rule 12 - Non-Subversion Rule

- Low-level access cannot bypass integrity rules
- All database access must go through RDBMS
- Security and integrity constraints always enforced

Relational Integrity Constraints

Q3: Explain different types of relational integrity constraints with examples.

Answer:

1. Domain Constraints

- Define valid values for attributes

- Enforced using data types and CHECK conditions
- Prevent invalid comparisons in queries
- Example: `age INT CHECK (age >= 18 AND age <= 120)`
- Most basic form of constraint checked during insert/update

2. Key Constraints

- Ensure uniqueness and non-nullability of primary keys
- Prevent duplicate or null entries for primary key attributes
- Part of entity integrity constraint
- Example: `Student(student_id PRIMARY KEY, name, dept)`
- Automatically enforced by DBMS during data operations

3. Entity Integrity

- Primary key values must be unique and NOT NULL
- Ensures every entity has unique identity
- Guarantees reliable foreign key references
- Example: `Employee(emp_id, name, salary)` where `emp_id` cannot be NULL
- Foundation for maintaining entity identification

4. Referential Integrity

- Foreign keys must match existing primary keys in referenced table
- Maintains consistency between related tables
- Prevents orphan records and dangling references
- Example: `Order(order_id, customer_id REFERENCES Customer(customer_id))`
- Can specify CASCADE, SET NULL, or RESTRICT on updates/deletes

5. Additional Constraints

- Functional dependencies used in normalization
- Triggers for complex business rules
- Assertions for database-wide constraints
- Check constraints for attribute-level validation
- Custom constraints for specific business requirements

Keys in Relational Model

Q4: Differentiate between superkey, candidate key, and primary key with examples.

Answer:

1. Superkey Definition and Properties

- Any set of attributes that uniquely identifies tuples
- May contain extra (redundant) attributes
- Not necessarily minimal set
- Foundation for understanding other key types
- Multiple superkeys possible for single relation

2. Superkey Examples

Employee(ID, Name, Phone, SSN, Email)
 Superkeys: {ID}, {SSN}, {Email}, {ID, Name},
 {SSN, Phone}, {ID, Name, Phone}, etc.

3. Candidate Key Definition and Properties

- Minimal superkey with no extraneous attributes
- Every candidate key uniquely identifies tuples
- No proper subset is also a superkey
- Relation may have multiple candidate keys
- Basis for choosing primary key

4. Candidate Key Examples

Student(ID, Email, Phone, Name)
 If all are unique: {ID}, {Email}, {Phone} are candidate keys
 {ID, Name} is NOT candidate key (contains extra attribute)

5. Primary Key Definition and Properties

- One candidate key chosen as main identifier
- Must be unique and NOT NULL
- Only one primary key per relation
- Referenced by foreign keys in other relations
- Should be stable (values don't change frequently)

6. Key Comparison Table

Feature	Superkey	Candidate Key	Primary Key
Uniqueness	Yes	Yes	Yes
Minimality	Not required	Required	Required
NULL allowed	Depends	No	No
Number per relation	Many	Many	Only one
Used for references	Possible	Possible	Yes

Normalization Process

Q5: Explain the purpose of normalization and discuss different types of anomalies with examples.

Answer:

1. Purpose of Normalization

- Organize data to minimize redundancy and inconsistency
- Prevent data anomalies through proper table structure
- Improve data integrity and reduce storage space
- Facilitate easier maintenance and updates
- Establish clear relationships between entities

2. Insertion Anomaly

- Cannot insert data due to missing values in required fields
- Example: Cannot add new department without assigning employee
- Occurs when unrelated data is stored together
- Solution: Separate independent entities into different tables
- Prevents incomplete information from blocking data entry

3. Update Anomaly

- Same information stored in multiple places becomes inconsistent
- Example: Employee address changes but updated in only some rows
- Leads to data inconsistency and contradictory information
- Requires multiple updates for single logical change
- Solution: Store information in single location only

4. Deletion Anomaly

- Removing record causes unintentional loss of valuable information
- Example: Deleting last employee in department removes department info
- Important data lost due to deletion of unrelated information
- Creates gaps in organizational knowledge

- Solution: Separate different types of information into appropriate tables

5. Benefits of Proper Normalization

- Eliminates data redundancy and reduces storage requirements
- Ensures data consistency and integrity
- Simplifies database maintenance and updates
- Reduces risk of data corruption
- Improves query performance for specific operations
- Creates flexible database structure for future modifications

Functional Dependencies Types

Q6: Explain different types of functional dependencies with examples.

Answer:

1. Basic Functional Dependency

- Attribute B functionally dependent on A if A determines B
- Notation: $A \rightarrow B$ (A functionally determines B)
- For any two tuples with same A value, B values must be identical
- Example: $\text{StudentID} \rightarrow \text{StudentName}, \text{StudentAge}$
- Foundation concept for database normalization

2. Trivial Functional Dependency

- Dependent attributes are subset of determinant
- Always holds true by definition
- Form: $X \rightarrow Y$ where $Y \subseteq X$
- Example: $\{\text{StudentID}, \text{Name}\} \rightarrow \text{StudentID}$
- Provides no useful information for normalization

3. Non-trivial Functional Dependency

- Dependent attributes not part of determinant
- Carries meaningful information about data relationships
- Form: $X \rightarrow Y$ where $Y \not\subseteq X$
- Example: $\text{StudentID} \rightarrow \text{StudentName}$ (Name not part of StudentID)
- Used in normalization process to identify problems

4. Completely Non-trivial Functional Dependency

- No overlap between determinant and dependent attributes
- Form: $X \rightarrow Y$ where $X \cap Y = \emptyset$ (empty set)

- Example: EmployeeID → EmployeeEmail
- Represents pure dependency relationships

5. Full Functional Dependency

- Dependent attribute requires entire composite key
- Cannot be determined by any proper subset of key
- Example: {SupplierID, ItemID} → Price (needs both attributes)
- Critical for achieving Second Normal Form (2NF)

6. Partial Functional Dependency

- Non-key attribute depends on part of composite key
- Violates Second Normal Form requirements
- Example: {StudentID, CourseID} → StudentName (depends only on StudentID)
- Must be eliminated to achieve 2NF

7. Transitive Functional Dependency

- Indirect dependency through intermediate attribute
- Form: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$
- Example: EmployeeID → DepartmentID → DepartmentName
- Must be eliminated to achieve Third Normal Form (3NF)

Normal Forms

Q7: Explain First, Second, and Third Normal Forms with examples.

Answer:

1. First Normal Form (1NF)

- **Rule:** All attributes must contain atomic (indivisible) values
- No multivalued or composite attributes allowed
- Each cell contains single value only
- Foundation for higher normal forms

Example Violation:

Student Table (Not in 1NF):

StudentID | Name | Courses

101 | John | Math, Physics, Chemistry

102 | Alice | Biology, Chemistry

1NF Solution:

Student Table (In 1NF):

StudentID | Name | Course

101	John	Math
101	John	Physics
101	John	Chemistry
102	Alice	Biology
102	Alice	Chemistry

2. Second Normal Form (2NF)

- **Rules:** Must be in 1NF AND no partial dependencies
- Non-prime attributes must depend on entire candidate key
- Eliminates redundancy caused by partial dependencies

Example with Partial Dependency:

Enrollment Table (1NF but not 2NF):

StudentID | CourseID | StudentName | CourseName | Grade

101	CS101	John	Database	A
101	CS102	John	Networks	B
102	CS101	Alice	Database	A

Problem: StudentName depends only on StudentID (partial dependency)

2NF Solution:

Students Table:

StudentID | StudentName

101 | John

102 | Alice

Enrollment Table:

StudentID | CourseID | Grade

101 | CS101 | A

101 | CS102 | B

102 | CS101 | A

Courses Table:

CourseID | CourseName

CS101 | Database

CS102 | Networks

3. Third Normal Form (3NF)

- **Rules:** Must be in 2NF AND no transitive dependencies
- Non-prime attributes should not depend on other non-prime attributes
- Eliminates redundancy from transitive relationships

Example with Transitive Dependency:

Employee Table (2NF but not 3NF):

EmpID | EmpName | DeptID | DeptName | DeptLocation

101 | John | D1 | HR | Building A

102 | Alice | D2 | IT | Building B

103 | Bob | D1 | HR | Building A

Problem: DeptName and DeptLocation depend on DeptID (transitive dependency)

3NF Solution:

Employee Table:

Department Table:

EmpID | EmpName | DeptID | DeptID | DeptName | DeptLocation

101 | John | D1 | D1 | HR | Building A

102 | Alice | D2 | D2 | IT | Building B

103 | Bob | D1

4. Benefits of Each Normal Form

- **1NF:** Eliminates repeating groups, enables relational operations
- **2NF:** Reduces redundancy from partial dependencies
- **3NF:** Eliminates most redundancy, suitable for most applications
- Each higher form builds upon previous forms
- Progressive reduction in data anomalies and redundancy

Armstrong's Axioms

Q8: Explain Armstrong's Axioms with examples and show how to compute attribute closure.

Answer:

1. Armstrong's Axioms Overview

- Set of inference rules for deriving functional dependencies
- Used to find closure of functional dependencies (F^+)
- Provides complete and sound system for FD inference
- Foundation for normalization algorithms
- Essential for database design verification

2. Primary Axioms (Basic Rules)

- **Reflexive Rule:** If $Y \subseteq X$, then $X \rightarrow Y$ Example: $\{A, B, C\} \rightarrow \{A, B\}$ (always true)
- **Augmentation Rule:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$ for any Z Example: If $A \rightarrow B$, then $AC \rightarrow BC$
- **Transitivity Rule:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$ Example: If $A \rightarrow B$ and $B \rightarrow C$, then $A \rightarrow C$

3. Secondary Axioms (Derived Rules)

- **Union Rule:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$ Example: If $A \rightarrow B$ and $A \rightarrow C$, then $A \rightarrow BC$
- **Decomposition Rule:** If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$ Example: If $A \rightarrow BC$, then $A \rightarrow B$ and $A \rightarrow C$
- **Pseudo-transitivity:** If $X \rightarrow Y$ and $YW \rightarrow Z$, then $XW \rightarrow Z$ Example: If $A \rightarrow B$ and $BC \rightarrow D$, then $AC \rightarrow D$

4. Attribute Closure Algorithm

- Purpose: Find all attributes determined by given set
- Input: Attribute set X , functional dependency set F
- Output: X^+ (closure of X)

Algorithm Steps:

```

1. Initialize: result := X
2. Repeat until no change:
   For each FD ( $Y \rightarrow Z$ ) in  $F$ :
     If  $Y \subseteq \text{result}$ , then  $\text{result} := \text{result} \cup Z$ 
3. Return result as  $X^+$ 

```

5. Attribute Closure Example Given: $R(A, B, C, D, E)$, $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D, D \rightarrow E\}$ Find: A^+ **Step-by-step Computation:**

```

Initial:  $A^+ = \{A\}$ 
Apply  $A \rightarrow B$ :  $A^+ = \{A, B\}$ 
Apply  $B \rightarrow C$ :  $A^+ = \{A, B, C\}$ 
Apply  $AC \rightarrow D$ :  $A^+ = \{A, B, C, D\}$  (since  $A, C \subseteq \{A, B, C\}$ )
Apply  $D \rightarrow E$ :  $A^+ = \{A, B, C, D, E\}$ 

```

Result: $A^+ = \{A, B, C, D, E\}$, so A is a superkey

Decomposition

Q9: Explain lossless-join decomposition and dependency preservation with examples.

Answer:

1. Lossless-Join Decomposition Definition

- Decomposition where original relation can be perfectly reconstructed
- Join of decomposed relations produces exact original relation
- No information is lost during decomposition process
- Critical property for maintaining data integrity
- Essential requirement for proper database design

2. Lossless-Join Condition

- Common attributes between decomposed relations must form superkey
- At least one decomposed relation must have common attributes as superkey
- Mathematical test: $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$
- Ensures unique reconstruction through natural join
- Prevents creation of spurious tuples during join

3. Lossless Decomposition Example

Original: $R(A, B, C)$ with FD: $A \rightarrow B$

Decomposition: $R_1(A, B), R_2(A, C)$

Test: Common attribute = A

Since $A \rightarrow B$ exists, A is superkey in R_1

Therefore, decomposition is lossless

Original Data: After Join:

A	B	C		A	B	C
1	2	3	→	1	2	3
4	5	6		4	5	6

4. Lossy Decomposition Example

Original: $R(A, B, C)$ with FD: $A \rightarrow B$

Decomposition: $R_1(A, B), R_2(B, C)$

Test: Common attribute = B

B is not superkey in either relation

Therefore, decomposition is lossy

May produce spurious tuples during join

5. Dependency Preservation Definition

- All original functional dependencies can be checked on decomposed relations
- No need to join relations to verify constraints
- Each FD is preserved in at least one decomposed relation
- Maintains integrity constraint enforcement
- Enables local constraint checking

6. Dependency Preservation Example

Original: R(Student_ID, Name, Course, Dept, Instructor)

FDs: Student_ID → Name, Course, Dept

Course → Instructor

Decomposition:

R₁(Student_ID, Name, Course, Dept)

R₂(Course, Instructor)

Analysis:

- FD₁: Student_ID → Name, Course, Dept (preserved in R₁)

- FD₂: Course → Instructor (preserved in R₂)

- All dependencies preserved locally

7. Trade-offs in Decomposition

- Lossless decomposition may not preserve all dependencies
- Dependency preservation may require lossy decomposition
- Goal: Achieve both properties when possible
- Sometimes must choose between conflicting requirements
- 3NF guarantees both properties, BCNF may sacrifice dependency preservation

Query Processing

Q10: Explain the phases of query processing with cost factors.

Answer:

1. Query Processing Overview

- Systematic approach to execute database queries efficiently
- Transforms high-level SQL into efficient execution plan
- Minimizes resource usage and execution time
- Three main phases: parsing, optimization, evaluation
- Critical for database performance and scalability

2. Phase 1: Parsing and Translation

- **Lexical Analysis:** Break SQL into tokens (keywords, identifiers, operators)
- **Syntax Analysis:** Check grammatical structure, detect syntax errors
- **Semantic Analysis:** Validate table/column existence, data type compatibility
- **Translation:** Convert to internal representation (relational algebra)
- Output: Logical query tree or algebraic expression

3. Phase 2: Query Optimization

- Generate multiple equivalent execution plans
- Apply transformation rules to improve efficiency
- Consider available indexes, statistics, and system resources
- Two approaches: heuristic (rule-based) and cost-based optimization
- Select plan with lowest estimated execution cost

4. Phase 3: Query Evaluation

- Execute chosen plan using database engine
- Map logical operations to physical algorithms
- Access data using appropriate methods (index scan, table scan)
- Apply join algorithms (nested loop, hash join, sort-merge)
- Return results to user application

5. Query Cost Factors

- **Disk I/O Cost:** Block transfers ($b \times t_t$) + Seeks ($s \times t_s$)
- **CPU Cost:** Processing time for comparisons and calculations
- **Memory Usage:** Buffer requirements and memory allocation
- **Network Cost:** Data transmission in distributed systems
- **Selectivity:** Fraction of tuples satisfying conditions

6. Cost Calculation Example

Selection Cost = $b \times t_t + s \times t_s$

Where:

b = number of block transfers

s = number of disk seeks

t_t = time per block transfer

t_s = time per seek

File Scan: Cost = $b_r \times t_t + 1 \times t_s$ (worst case)

Index Scan: Cost = $(h_i + 1) \times (t_t + t_s)$

7. Optimization Techniques

- **Heuristic Rules:** Apply selections early, use joins instead of cartesian products
 - **Cost Estimation:** Calculate costs for different access methods
 - **Join Ordering:** Determine optimal sequence for multiple joins
 - **Index Usage:** Choose appropriate indexes for query conditions
 - **Materialization vs Pipelining:** Decide on intermediate result handling
-

Summary

This structured question bank covers all major topics in Unit 2 with:

- **15 Short Answer Questions** (2-3 marks each)
- **10 Long Answer Questions** (5-8 marks each)
- Comprehensive coverage of relational model, integrity constraints, normalization, and query processing
- Each question includes detailed explanations with examples
- Logical progression from basic concepts to advanced topics
- Practical examples and real-world applications
- Clear formatting and structured presentation